

Lesson 02 Demo 02

Executing Serial and Parallel Workflows Using CrewAI Agents

Objective: To demonstrate how to build intelligent workflows using CrewAI agents that execute tasks either sequentially (serial) or independently (parallel)

This demo highlights how tasks can be structured based on interdependence or concurrency.

Scenario: A product leader at a global FMCG company requires timely insights on how AI is reshaping product development, consumer engagement, and supply chain efficiency. Manually gathering and summarizing this information is time-consuming and slows product strategy decisions. To solve this, the workflow can be automated with AI agents—one dedicated to researching the latest AI trends in FMCG and another to generating executive-ready summaries. This enables the product team to make faster, data-informed decisions in a highly competitive market.

Tools required: Visual Studio Code

Prerequisites: Azure OpenAI API key

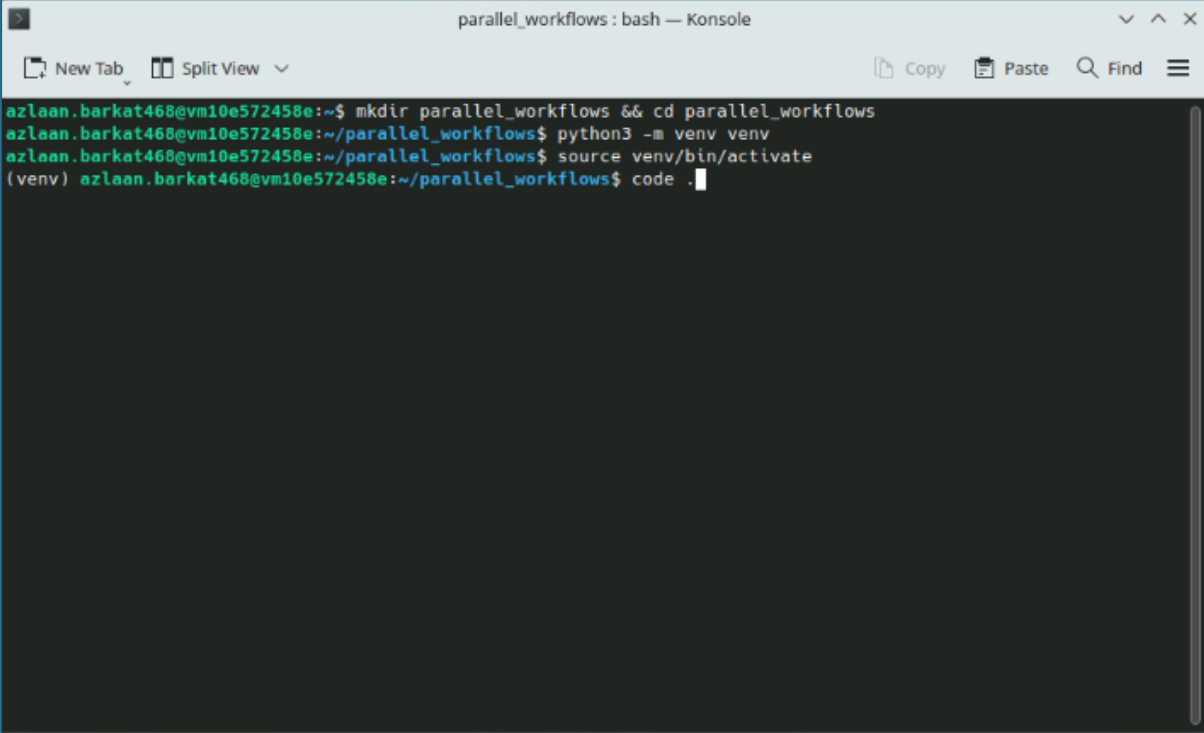
Steps to be followed:

1. Set up your local environment in Visual Studio Code
2. Configure the Tavily API Key and Azure OpenAI model
3. Define agents and tasks
4. Define and run parallel tasks

Step 1: Set up your local environment in Visual Studio Code

1.1 Open the terminal in your system and run the following commands:

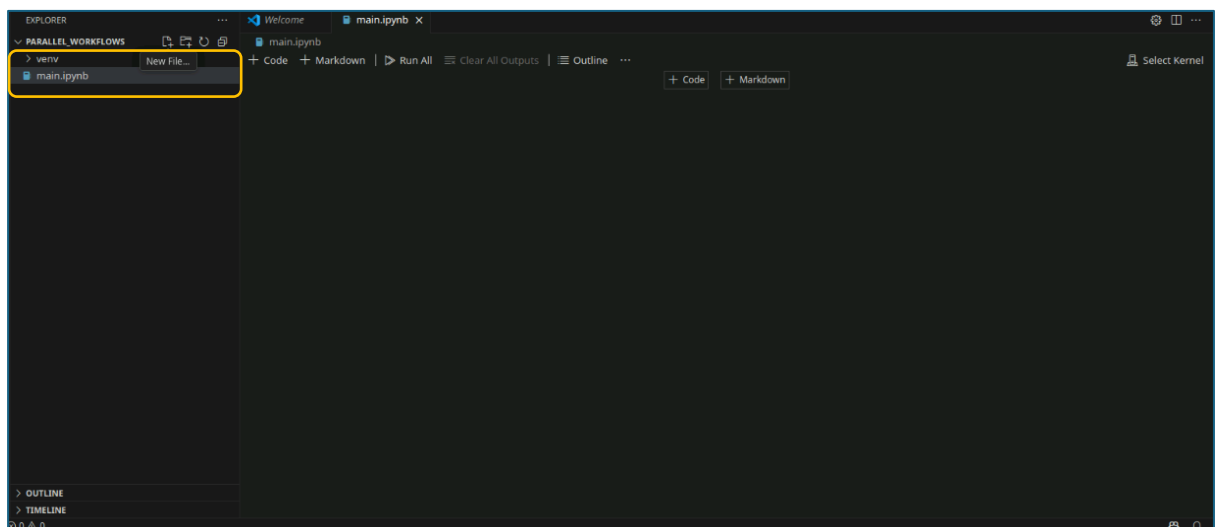
```
mkdir parallel_workflows && cd parallel_workflows
python3 -m venv venv
source venv/bin/activate
code .
```



A terminal window titled "parallel_workflows : bash — Konsole" showing the execution of the following commands:

```
azlaan.barkat468@vm10e572458e:~$ mkdir parallel_workflows && cd parallel_workflows
azlaan.barkat468@vm10e572458e:~/parallel_workflows$ python3 -m venv venv
azlaan.barkat468@vm10e572458e:~/parallel_workflows$ source venv/bin/activate
(venv) azlaan.barkat468@vm10e572458e:~/parallel_workflows$ code .
```

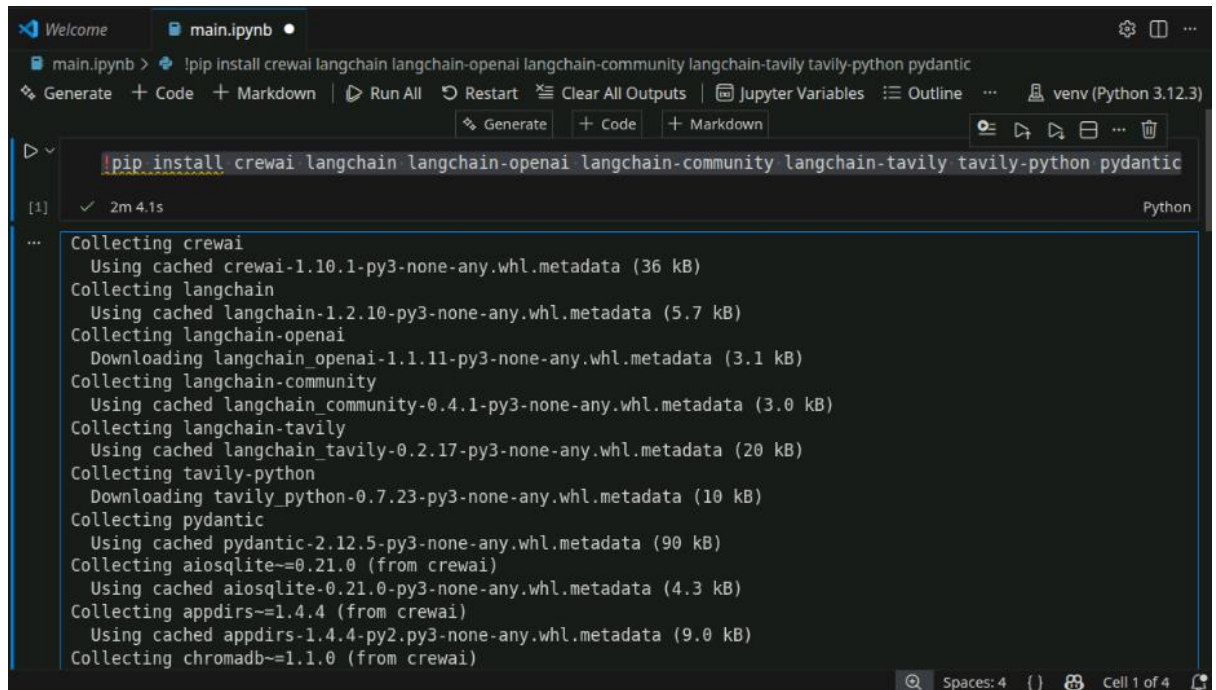
1.2 Create a new file: **main.ipynb**



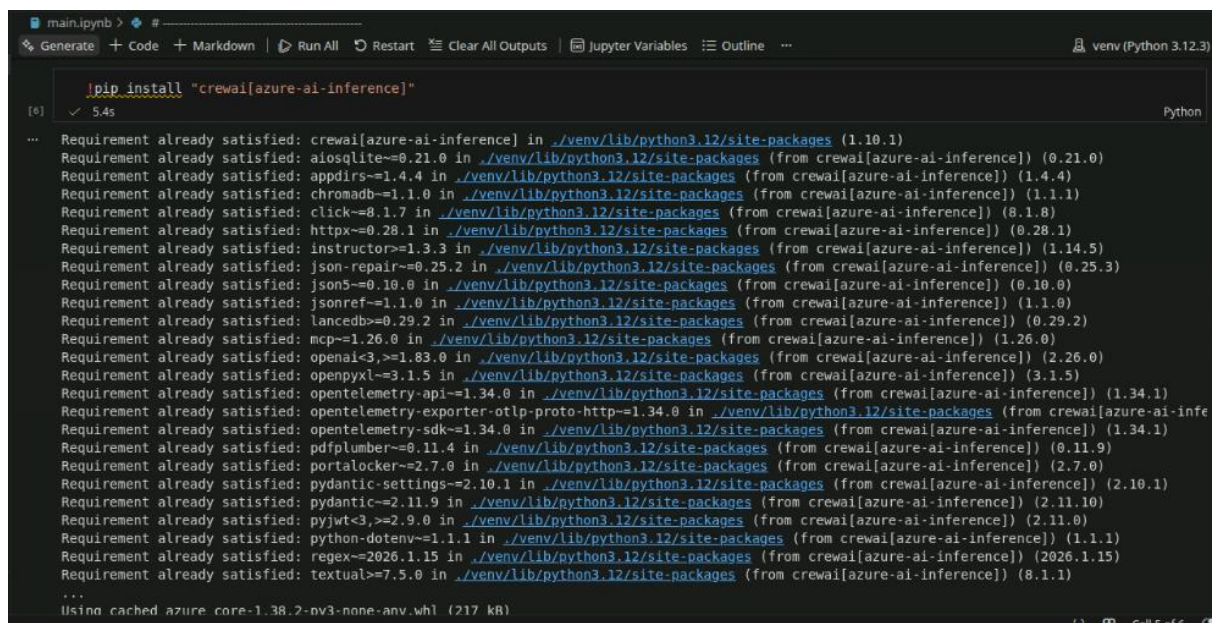
1.3 In the first 2 cells, write and run the following commands to install the required packages:

!pip install crewai langchain langchain-openai langchain-community langchain-tavily tavily-python pydantic

!pip install "crewai[azure-ai-inference]"



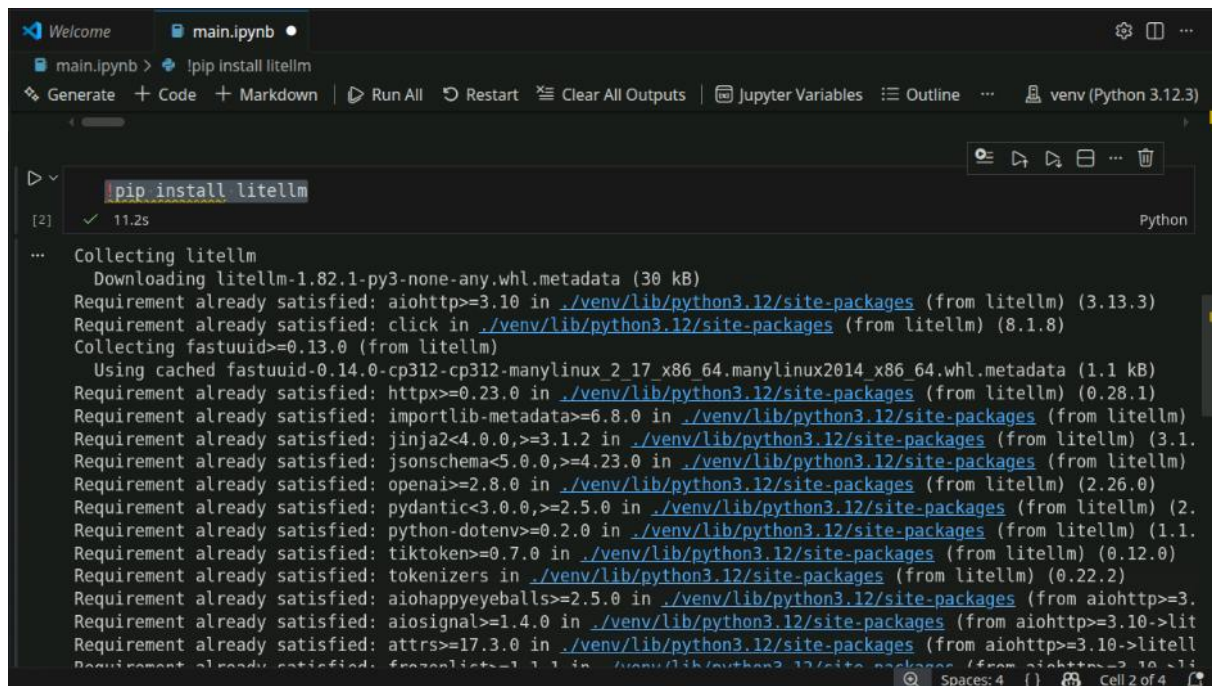
```
main.ipynb •
main.ipynb > !pip install crewai langchain langchain-openai langchain-community langchain-tavily tavily-python pydantic
Generate + Code + Markdown | Run All Restart Clear All Outputs Jupyter Variables Outline ... venv (Python 3.12.3)
Generate + Code + Markdown
[1] ✓ 2m 4.1s Python
Collecting crewai
  Using cached crewai-1.10.1-py3-none-any.whl.metadata (36 kB)
Collecting langchain
  Using cached langchain-1.2.10-py3-none-any.whl.metadata (5.7 kB)
Collecting langchain-openai
  Downloading langchain_openai-1.1.11-py3-none-any.whl.metadata (3.1 kB)
Collecting langchain-community
  Using cached langchain_community-0.4.1-py3-none-any.whl.metadata (3.0 kB)
Collecting langchain-tavily
  Using cached langchain_tavily-0.2.17-py3-none-any.whl.metadata (20 kB)
Collecting tavily-python
  Downloading tavily_python-0.7.23-py3-none-any.whl.metadata (10 kB)
Collecting pydantic
  Using cached pydantic-2.12.5-py3-none-any.whl.metadata (90 kB)
Collecting aiosqlite~=0.21.0 (from crewai)
  Using cached aiosqlite-0.21.0-py3-none-any.whl.metadata (4.3 kB)
Collecting appdirs~=1.4.4 (from crewai)
  Using cached appdirs-1.4.4-py2.py3-none-any.whl.metadata (9.0 kB)
Collecting chromadb~=1.1.0 (from crewai)
```



```
main.ipynb > #
main.ipynb > !pip install "crewai[azure-ai-inference]"
Generate + Code + Markdown | Run All Restart Clear All Outputs Jupyter Variables Outline ... venv (Python 3.12.3)
[6] ✓ 5.4s Python
Requirement already satisfied: crewai[azure-ai-inference] in ./venv/lib/python3.12/site-packages (1.10.1)
Requirement already satisfied: aiosqlite~=0.21.0 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (0.21.0)
Requirement already satisfied: appdirs~=1.4.4 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (1.4.4)
Requirement already satisfied: chromadb~=1.1.0 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (1.1.1)
Requirement already satisfied: click~=8.1.7 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (8.1.8)
Requirement already satisfied: httpx~=0.28.1 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (0.28.1)
Requirement already satisfied: instructor~=1.3.3 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (1.14.5)
Requirement already satisfied: json-repair~=0.25.2 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (0.25.3)
Requirement already satisfied: json5~=0.10.0 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (0.10.0)
Requirement already satisfied: jsonref~=1.1.0 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (1.1.0)
Requirement already satisfied: lance~=0.29.2 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (0.29.2)
Requirement already satisfied: mcp~=1.26.0 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (1.26.0)
Requirement already satisfied: openai<3,>=1.83.0 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (2.12.0)
Requirement already satisfied: openpyxl~=3.1.5 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (3.1.5)
Requirement already satisfied: opentelemetry-api~=1.34.0 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (1.34.1)
Requirement already satisfied: opentelemetry-exporter-otlp-proto-http~=1.34.0 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (1.34.1)
Requirement already satisfied: opentelemetry-sdk~=1.34.0 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (1.34.1)
Requirement already satisfied: pdfplumber~=0.11.4 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (0.11.9)
Requirement already satisfied: portalocker~=2.7.0 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (2.7.0)
Requirement already satisfied: pydantic-settings~=2.10.1 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (2.10.1)
Requirement already satisfied: pydantic~=2.11.9 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (2.11.10)
Requirement already satisfied: pyjwt<3,>=2.9.0 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (2.11.0)
Requirement already satisfied: python-dotenv~=1.1.1 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (1.1.1)
Requirement already satisfied: regex~=2026.1.15 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (2026.1.15)
Requirement already satisfied: textual~=7.5.0 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (8.1.1)
...
Using cached azure_core-1.38.2-py3-none-any.whl (217 kB)
```

1.4 In the next cell, write the following command to install litellm:

!pip install litellm



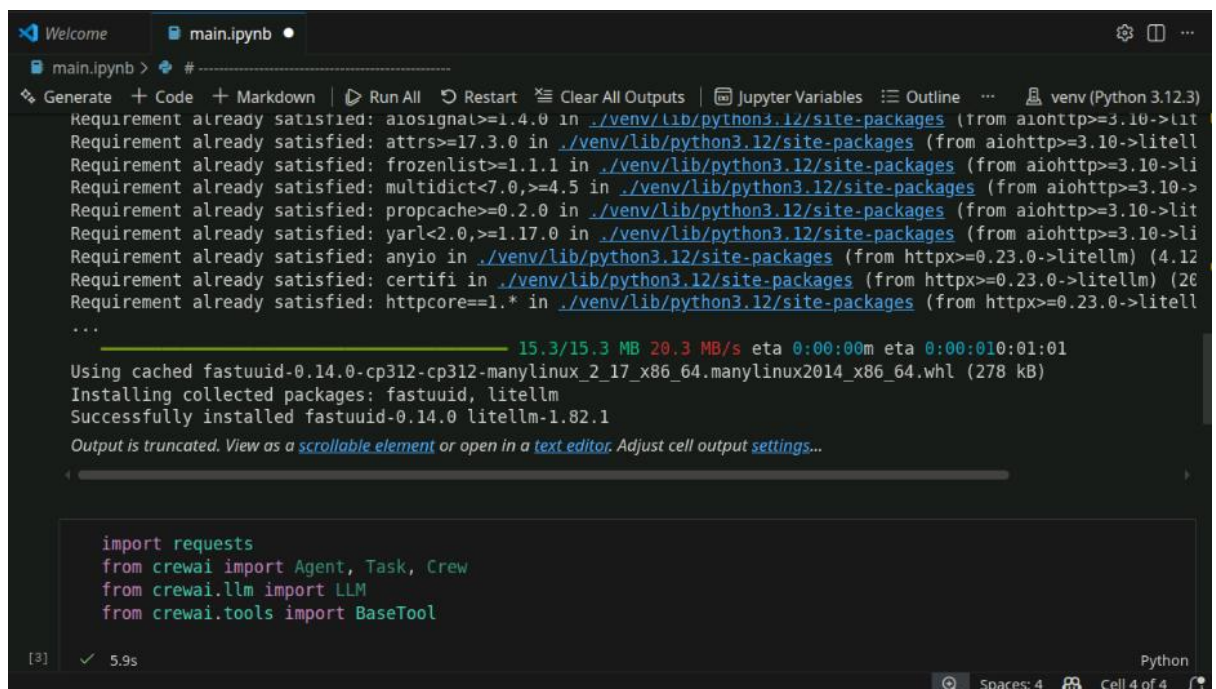
```
main.ipynb •
main.ipynb > !pip install litellm
Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline ... venv (Python 3.12.3)

[2] ✓ 11.2s Python

... Collecting litellm
  Downloading litellm-1.82.1-py3-none-any.whl.metadata (30 kB)
  Requirement already satisfied: aiohttp>=3.10 in ./venv/lib/python3.12/site-packages (from litellm) (3.13.3)
  Requirement already satisfied: click in ./venv/lib/python3.12/site-packages (from litellm) (8.1.8)
  Collecting fastuuid>=0.13.0 (from litellm)
  Using cached fastuuid-0.14.0-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (1.1 kB)
  Requirement already satisfied: httpx>=0.23.0 in ./venv/lib/python3.12/site-packages (from litellm) (0.28.1)
  Requirement already satisfied: importlib-metadata>=6.8.0 in ./venv/lib/python3.12/site-packages (from litellm) (7.1.0)
  Requirement already satisfied: jinja2<4.0.0,>=3.1.2 in ./venv/lib/python3.12/site-packages (from litellm) (3.1.3)
  Requirement already satisfied: jsonschema<5.0.0,>=4.23.0 in ./venv/lib/python3.12/site-packages (from litellm) (4.23.0)
  Requirement already satisfied: openai>=2.8.0 in ./venv/lib/python3.12/site-packages (from litellm) (2.26.0)
  Requirement already satisfied: pydantic<3.0.0,>=2.5.0 in ./venv/lib/python3.12/site-packages (from litellm) (2.10.6)
  Requirement already satisfied: python-dotenv>=0.2.0 in ./venv/lib/python3.12/site-packages (from litellm) (1.1.0)
  Requirement already satisfied: tiktoken>=0.7.0 in ./venv/lib/python3.12/site-packages (from litellm) (0.12.0)
  Requirement already satisfied: tokenizers in ./venv/lib/python3.12/site-packages (from litellm) (0.22.2)
  Requirement already satisfied: aiohappyeyeballs>=2.5.0 in ./venv/lib/python3.12/site-packages (from aiohttp>=3.10->litellm) (2.5.0)
  Requirement already satisfied: aiosignal>=1.4.0 in ./venv/lib/python3.12/site-packages (from aiohttp>=3.10->litellm) (1.4.0)
  Requirement already satisfied: attrs>=17.3.0 in ./venv/lib/python3.12/site-packages (from aiohttp>=3.10->litellm) (25.1.0)
  Requirement already satisfied: frozenlist>=1.1.1 in ./venv/lib/python3.12/site-packages (from aiohttp>=3.10->litellm) (1.4.1)
  Requirement already satisfied: multidict<7.0,>=4.5 in ./venv/lib/python3.12/site-packages (from aiohttp>=3.10->litellm) (6.0.5)
  Requirement already satisfied: propcache>=0.2.0 in ./venv/lib/python3.12/site-packages (from aiohttp>=3.10->litellm) (0.2.0)
  Requirement already satisfied: yarl<2.0,>=1.17.0 in ./venv/lib/python3.12/site-packages (from aiohttp>=3.10->litellm) (1.17.0)
  Requirement already satisfied: anyio in ./venv/lib/python3.12/site-packages (from httpx>=0.23.0->litellm) (4.12.0)
  Requirement already satisfied: certifi in ./venv/lib/python3.12/site-packages (from httpx>=0.23.0->litellm) (2025.1.1)
  Requirement already satisfied: httpcore==1.* in ./venv/lib/python3.12/site-packages (from httpx>=0.23.0->litellm) (1.0.7)
  Requirement already satisfied: idna in ./venv/lib/python3.12/site-packages (from httpx>=0.23.0->litellm) (3.10)
  Requirement already satisfied: sniffio in ./venv/lib/python3.12/site-packages (from httpx>=0.23.0->litellm) (1.3.1)
  Requirement already satisfied: typing-extensions in ./venv/lib/python3.12/site-packages (from httpx>=0.23.0->litellm) (4.12.2)
  Requirement already satisfied: urllib3 in ./venv/lib/python3.12/site-packages (from httpx>=0.23.0->litellm) (2.3.0)
  Requirement already satisfied: zipp in ./venv/lib/python3.12/site-packages (from importlib-metadata->litellm) (3.19.2)
  Installing collected packages: fastuuid, litellm
  Successfully installed fastuuid-0.14.0 litellm-1.82.1
  Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

1.5 Import the required libraries using the following code:

```
import requests
from crewai import Agent, Task, Crew
from crewai.llm import LLM
from crewai.tools import BaseTool
```



```
main.ipynb •
main.ipynb > # -----
Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline ... venv (Python 3.12.3)

Requirement already satisfied: aiosignal>=1.4.0 in ./venv/lib/python3.12/site-packages (from aiohttp>=3.10->litellm) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in ./venv/lib/python3.12/site-packages (from aiohttp>=3.10->litellm) (25.1.0)
Requirement already satisfied: frozenlist>=1.1.1 in ./venv/lib/python3.12/site-packages (from aiohttp>=3.10->litellm) (1.4.1)
Requirement already satisfied: multidict<7.0,>=4.5 in ./venv/lib/python3.12/site-packages (from aiohttp>=3.10->litellm) (6.0.5)
Requirement already satisfied: propcache>=0.2.0 in ./venv/lib/python3.12/site-packages (from aiohttp>=3.10->litellm) (0.2.0)
Requirement already satisfied: yarl<2.0,>=1.17.0 in ./venv/lib/python3.12/site-packages (from aiohttp>=3.10->litellm) (1.17.0)
Requirement already satisfied: anyio in ./venv/lib/python3.12/site-packages (from httpx>=0.23.0->litellm) (4.12.0)
Requirement already satisfied: certifi in ./venv/lib/python3.12/site-packages (from httpx>=0.23.0->litellm) (2025.1.1)
Requirement already satisfied: httpcore==1.* in ./venv/lib/python3.12/site-packages (from httpx>=0.23.0->litellm) (1.0.7)
Requirement already satisfied: idna in ./venv/lib/python3.12/site-packages (from httpx>=0.23.0->litellm) (3.10)
Requirement already satisfied: sniffio in ./venv/lib/python3.12/site-packages (from httpx>=0.23.0->litellm) (1.3.1)
Requirement already satisfied: typing-extensions in ./venv/lib/python3.12/site-packages (from httpx>=0.23.0->litellm) (4.12.2)
Requirement already satisfied: urllib3 in ./venv/lib/python3.12/site-packages (from httpx>=0.23.0->litellm) (2.3.0)
Requirement already satisfied: zipp in ./venv/lib/python3.12/site-packages (from importlib-metadata->litellm) (3.19.2)
Installing collected packages: fastuuid, litellm
Successfully installed fastuuid-0.14.0 litellm-1.82.1
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

import requests
from crewai import Agent, Task, Crew
from crewai.llm import LLM
from crewai.tools import BaseTool

[3] ✓ 5.9s Python
```

Step 2: Configure the Tavily API Key and Azure OpenAI model

2.1 Use the Tavily Key and Azure OpenAI API key to build an LLM using the following code:

```
# -----  
# Hardcoded credentials  
# -----  
AZURE_OPENAI_API_KEY =  
"2ABecnfzxhRg4M5D6pBKiqxXVhmGB2WvQ0aYKkbTCPsj0JLKsZPfJQQJ99BDAC77b  
zfXJ3w3AAABACOGi3sC"  
AZURE_OPENAI_ENDPOINT = "https://openai-api-management-gw.azure-api.net/"  
AZURE_OPENAI_API_VERSION = "2024-12-01-preview"  
AZURE_OPENAI_DEPLOYMENT = "gpt-5-mini"  
  
TAVILY_API_KEY = "tvly-dev-1uT9rc-  
X1i2A54uzUo18Hhz6BL1UjY8KMsU5aSCBbu3VRbNkr"  
  
# -----  
# Tavily Web Search Tool  
# -----  
class TavilySearchTool(BaseTool):  
    name: str = "web_search"  
    description: str = "Search the web for recent information."  
  
    def _run(self, query: str):  
        url = "https://api.tavily.com/search"  
        payload = {  
            "api_key": TAVILY_API_KEY,  
            "query": query,  
            "max_results": 3  
        }  
  
        response = requests.post(url, json=payload, timeout=30)  
        response.raise_for_status()  
        data = response.json()  
  
        results = []  
        for r in data.get("results", []):  
            title = r.get("title", "No title")  
            link = r.get("url", "No URL")  
            results.append(f"{title} - {link}")  
  
        return "\n".join(results) if results else "No web results found."
```

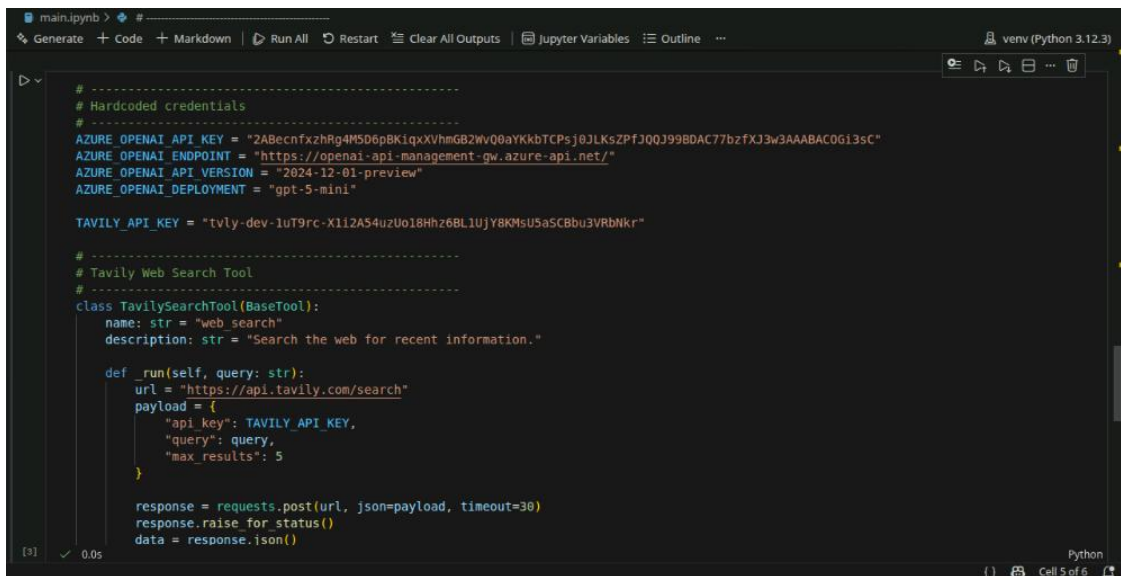
```
search_tool = TavilySearchTool()
```

```
# -----
```

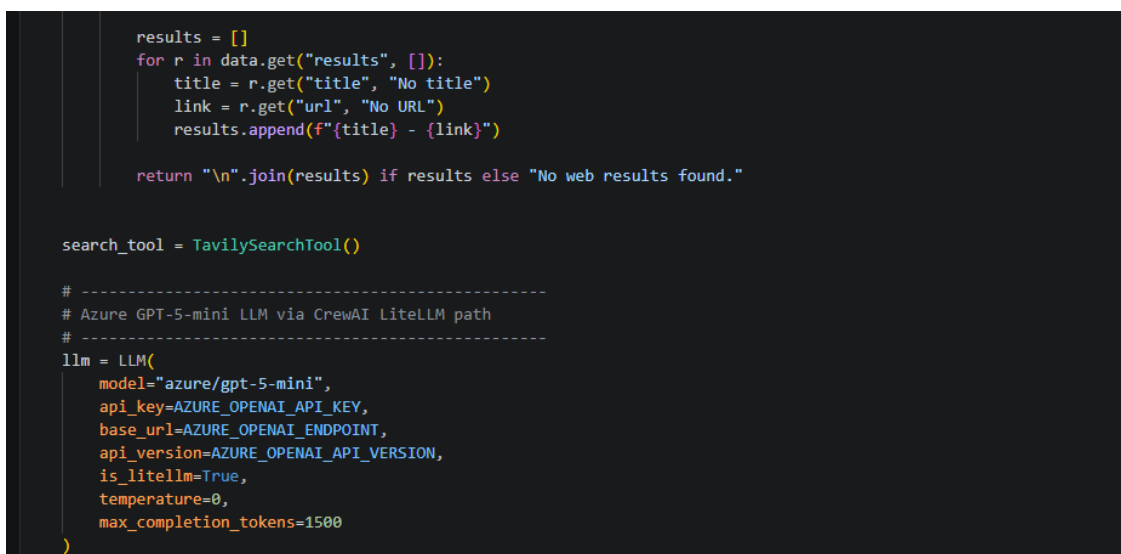
```
# Azure GPT-5-mini LLM via CrewAI LiteLLM path
```

```
# -----
```

```
llm = LLM(  
    model="azure/gpt-5-mini",  
    api_key=AZURE_OPENAI_API_KEY,  
    base_url=AZURE_OPENAI_ENDPOINT,  
    api_version=AZURE_OPENAI_API_VERSION,  
    is_litellm=True,  
    temperature=0,  
    max_completion_token=1500  
)
```



```
# -----  
# Hardcoded credentials  
# -----  
AZURE_OPENAI_API_KEY = "2ABecnfxzhRg4MSD6pBK1qxXVhmG82WvQ0aYKkbTCPsj0JLKsZPfJQ0J998DAC77bzfxJ3w3AAAABACOG13sC"  
AZURE_OPENAI_ENDPOINT = "https://openai-api-management-gw.azure-api.net/"  
AZURE_OPENAI_API_VERSION = "2024-12-01-preview"  
AZURE_OPENAI_DEPLOYMENT = "gpt-5-mini"  
  
TAVILY_API_KEY = "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"  
  
# -----  
# Tavily Web Search Tool  
# -----  
class TavilySearchTool(BaseTool):  
    name: str = "web_search"  
    description: str = "Search the web for recent information."  
  
    def _run(self, query: str):  
        url = "https://api.tavily.com/search"  
        payload = {  
            "api_key": TAVILY_API_KEY,  
            "query": query,  
            "max_results": 5  
        }  
  
        response = requests.post(url, json=payload, timeout=30)  
        response.raise_for_status()  
        data = response.json()  
  
[3]: ✓ 0.0s
```



```
        results = []  
        for r in data.get("results", []):  
            title = r.get("title", "No title")  
            link = r.get("url", "No URL")  
            results.append(f"{title} - {link}")  
  
        return "\n".join(results) if results else "No web results found."  
  
search_tool = TavilySearchTool()  
  
# -----  
# Azure GPT-5-mini LLM via CrewAI LiteLLM path  
# -----  
llm = LLM(  
    model="azure/gpt-5-mini",  
    api_key=AZURE_OPENAI_API_KEY,  
    base_url=AZURE_OPENAI_ENDPOINT,  
    api_version=AZURE_OPENAI_API_VERSION,  
    is_litellm=True,  
    temperature=0,  
    max_completion_tokens=1500  
)
```


Note: Generate your own Tavily Key and use it in the above code block.

Step 3: Define agents and tasks

3.1 Use the following code to create agents:

```
# -----  
# Agents  
# -----  
researcher = Agent(  
    role="AI Researcher",  
    goal="Find the latest advancements in AI for healthcare",  
    backstory=(  
        "You are an expert in artificial intelligence and stay updated "  
        "with the latest research trends in healthcare."  
    ),  
    verbose=True,  
    allow_delegation=False,  
    llm=llm,  
    max_iter=2,  
    tools=[search_tool]  
)  
  
writer = Agent(  
    role="Technical Writer",  
    goal="Summarize research into an executive report",  
    backstory=(  
        "You are an experienced technical writer with expertise in "  
        "summarizing healthcare research for executives."  
    ),  
    verbose=True,  
    allow_delegation=False,  
    llm=llm  
)
```

```

# -----
# Agents
# -----
researcher = Agent(
    role="AI Researcher",
    goal="Find the latest advancements in AI for healthcare",
    backstory=(
        "You are an expert in artificial intelligence and stay updated "
        "with the latest research trends in healthcare."
    ),
    verbose=True,
    allow_delegation=False,
    llm=llm,
    max_iter=2,
    tools=[search_tool]
)

writer = Agent(
    role="Technical Writer",
    goal="Summarize research into an executive report",
    backstory=(
        "You are an experienced technical writer with expertise in "
        "summarizing healthcare research for executives."
    ),
    verbose=True,
    allow_delegation=False,
    llm=llm
)

```

3.2 Define and run serial tasks using the following code:

```

# -----
# SERIAL EXECUTION
# -----
task_research = Task(
    description=(
        "Use the web search results to explain the top 3 recent advancements in AI for healthcare in 3-4 sentences."
        "Do not call tools again after getting results."
    ),
    expected_output=(
        "Detailed notes on three advancements, with names and explanations."
    ),
    agent=researcher
)

task_write = Task(
    description=(
        "Write a short executive summary using the research notes provided by the AI Researcher. "
        "Limit the answer to about 100 words."
    ),
    expected_output=(
        "An executive summary report of the top 3 AI advancements in healthcare."
    ),
    agent=writer,
    context=[task_research]
)

```



```

)

print("\n=== SERIAL EXECUTION ===")

crew_serial = Crew(
    agents=[researcher, writer],
    tasks=[task_research, task_write],
    verbose=True
)

serial_result = await crew_serial.kickoff_async()

print("\n[Serial Result]:\n")
try:
    print(serial_result.raw)
except AttributeError:
    print(serial_result)

```

```

# -----
# SERIAL EXECUTION
# -----
task_research = Task(
    description=(
        "Use the web search results to explain the top 3 recent advancements in AI for healthcare in 3-4 sentences."
        "Do not call tools again after getting results."
    ),
    expected_output=(
        "Detailed notes on three advancements, with names and explanations."
    ),
    agent=researcher
)

task_write = Task(
    description=(
        "Write a short executive summary using the research notes provided by the AI Researcher. "
        "Limit the answer to about 100 words."
    ),
    expected_output=(
        "An executive summary report of the top 3 AI advancements in healthcare."
    ),
    agent=writer,
    context=[task_research]
)

print("\n=== SERIAL EXECUTION ===")

```

```

task_write = Task(
    description=(
        "Write a short executive summary using the research notes provided by the AI Researcher. "
        "Limit the answer to about 100 words."
    ),
    expected_output=(
        "An executive summary report of the top 3 AI advancements in healthcare."
    ),
    agent=writer,
    context=[task_research]
)

print("\n=== SERIAL EXECUTION ===")

crew_serial = Crew(
    agents=[researcher, writer],
    tasks=[task_research, task_write],
    verbose=True
)

serial_result = await crew_serial.kickoff_async()

print("\n[Serial Result]:\n")
try:
    print(serial_result.raw)
except AttributeError:
    print(serial_result)

```

The program generates the following output showing serial execution:

```

...
=== SERIAL EXECUTION ===
...


Crew Execution Started



Name: crew



ID: f5fe313f-daf9-41af-9fbd-d3955b313c5d


...


Task Started



Name: Use the web search results to explain the top 3 recent advancements in AI for healthcare in 5-6 sentences.Do not call tools again after getting results.



ID: 70305978-e761-458e-b74e-dfb83e06a233


...


Agent Started


...

```

```

main.ipynb > # -----
Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline ...
venv (Python 3.12.3)

...
Agent: AI Researcher

Final Answer:

1) Foundation models and generative AI (large language models and multimodal models such as GPT-4 and Med-PaLM 2): these models are being adapted for clinical decision support, automated chart summarization, patient communication, coding/triage and multimodal interpretation (text, images, signals), bringing major gains in clinician efficiency and information access. They require domain-specific fine-tuning, rigorous validation, and deployment guardrails because risks like hallucinations, bias, and privacy leakage can lead to unsafe or inaccurate clinical outputs.

2) AI-accelerated drug discovery and protein structure prediction (driven by deep generative models and breakthroughs such as AlphaFold): generative chemistry, structure prediction, and in silico screening have dramatically shortened the timelines for target identification and lead generation, enabling virtual design of candidate molecules and better understanding of protein conformations. These methods lower early-stage costs and increase throughput for pharma and biotech, but outputs still need experimental validation and careful integration with ADMET/pharmacology pipelines.

3) Federated learning and privacy-preserving machine learning (secure aggregation, differential privacy, homomorphic encryption for multi-site training): these approaches let institutions collaboratively train robust, generalizable models on distributed EHR and imaging data without centralizing patient records, improving performance across diverse populations while meeting legal and privacy constraints. Adoption in radiology, pathology, and EHR modeling has improved model fairness and generalizability, though heterogeneity of data, communication overhead, and operational complexity remain challenges for broad clinical rollout.

...

```

```

main.ipynb > # -----
Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline ...
venv (Python 3.12.3)

...
[Serial Result]:

Executive Summary – Top 3 AI Advancements in Healthcare

1) Foundation models & generative AI
What: Large language and multimodal models (e.g., GPT-4, Med-PaLM 2) adapted for clinical decision support, automated chart summarization, patient communication, coding/triage and multimodal interpretation (text, images, signals), bringing major gains in clinician efficiency and information access, and improved patient engagement.
Impact: Major gains in clinician efficiency, faster information access, and improved patient engagement.
Risks/Needs: Hallucinations, bias, and privacy leakage require domain-specific fine-tuning, rigorous clinical validation, and deployment guardrails.

2) AI-accelerated drug discovery & protein structure prediction
What: Deep generative models and breakthroughs like AlphaFold enable generative chemistry, structure prediction, and high-throughput in silico screening have dramatically shortened the timelines for target identification and lead generation; better insight into protein conformation.
Impact: Shortened timelines and lower early-stage costs for target identification and lead generation; better insight into protein conformation.
Risks/Needs: Computational outputs require experimental validation and integration with ADMET/pharmacology pipelines to de-risk candidates.

3) Federated learning & privacy-preserving ML
What: Secure aggregation, differential privacy, and homomorphic encryption enable multi-site model training without centralizing patient data.
Impact: Improved model generalizability and fairness across diverse populations; early adoption in radiology, pathology, and EHR modeling.
Risks/Needs: Data heterogeneity, communication overhead, and operational complexity necessitate robust governance, standards, and infrastructure.

Priority recommendations: fund validation and governance frameworks, pilot federated collaborations, and form strategic partnerships to operationalize.

...
Crew Completion

Crew Execution Completed
Name: crew
ID: 25f6a13f-4a79-41a7-9fbd-d3955b313c5d
Final Output: Executive Summary – Top 3 AI Advancements in Healthcare

Spaces: 4 {} Cell 7 of 7

```

Step 4: Define and run parallel tasks

4.1 Now, define and run the parallel tasks using the following code:

```

# -----
# PARALLEL EXECUTION
# -----

task_parallel_1 = Task(
    description=(
        "Use web search to list 5 AI companies focusing on drug discovery. "
        "For each company, give one short line about what they specialize in."
    ),
    expected_output="Company names and what they specialize in.",

```

```

        async_execution=True,
        agent=researcher
    )

    task_parallel_2 = Task(
        description=(
            "Write a short report on how AI is transforming patient diagnostics. "
            "Limit the answer to about 200 words."
        ),
        expected_output="A short report with examples and explanation.",
        agent=writer
    )

    print("\n=== PARALLEL EXECUTION ===")

    crew_parallel = Crew(
        agents=[researcher, writer],
        tasks=[task_parallel_1, task_parallel_2],
        verbose=True
    )

    parallel_result = await crew_parallel.kickoff_async()

    print("\n[Parallel Result]:\n")
    try:
        print(parallel_result.raw)
    except AttributeError:
        print(parallel_result)

```

```
main.ipynb > #
Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline ...
venv (Python 3.12.3)

# -----
# PARALLEL EXECUTION
# -----
task_parallel_1 = Task(
    description=(
        "Use web search to list 5 AI companies focusing on drug discovery. "
        "For each company, give one short line about what they specialize in."
    ),
    expected_output="Company names and what they specialize in.",
    async_execution=True,
    agent=researcher
)

task_parallel_2 = Task(
    description=(
        "Write a short report on how AI is transforming patient diagnostics. "
        "Limit the answer to about 200 words."
    ),
    expected_output="A short report with examples and explanation.",
    agent=writer
)

print("\n=== PARALLEL EXECUTION ===")

crew_parallel = Crew(
    agents=[researcher, writer],
    tasks=[task_parallel_1, task_parallel_2],
    verbose=True
)

[6] Python
Spaces: 4 {} Cell 8 of 8
```

```
task_parallel_2 = Task(
    description=(
        "Write a short report on how AI is transforming patient diagnostics. "
        "Limit the answer to about 100 words."
    ),
    expected_output="A short report with examples and explanation.",
    agent=writer
)

print("\n=== PARALLEL EXECUTION ===")

crew_parallel = Crew(
    agents=[researcher, writer],
    tasks=[task_parallel_1, task_parallel_2],
    verbose=True
)

parallel_result = await crew_parallel.kickoff_async()

print("\n[Parallel Result]:\n")
try:
    print(parallel_result.raw)
except AttributeError:
    print(parallel_result)
```

The program generates the following output showing parallel execution:

```
main.ipynb > #
Generate + Code + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline ...
venv (Python 3.12.3)

...

=== PARALLEL EXECUTION ===

...

[ ] Crew Execution Started

Crew Execution Started
Name: crew
ID: 0ef75636-25d1-4f95-bc8b-12275bdc7967

...

[ ] Task Started

Task Started
Name: Use web search to list 5 AI companies focusing on drug discovery. For each company, give one short line
about what they specialize in.
ID: f5a2b2e4-eb30-4926-b9a6-a3e0e33aef63

...

[ ] Agent Started

Agent: AI Researcher

Spaces: 4 | () | Cell 8 of 8

main.ipynb > #
Generate + Code + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline ...
venv (Python 3.12.3)

...

Tool web_search executed with result: What Does Atomwise Do? AI-Powered Drug Discovery ... - https://www.promptloop.com/directory/what-does-atc
Atomwise - AI Solution Features, Pricing & Reviews - https://welcome.ai/solution/atomw...
Tool web_search executed with result: Exscientia: a clinical pipeline for AI-designed drug candidates - UKRI - https://www.ukri.org/who-we-are/
Tool web_search executed with result: AI-Native Drug Discovery using Insilico Medicine's Nach01 Model ... - https://techcommunity.microsoft.com
Tool web_search executed with result: Recursion: Pioneering AI Drug Discovery - https://www.recursion.com/
How Recursion Pharmaceuticals is Using AI to Revolutionize Drug ... - https://machine-learning-made-simple.medium.com/how-recursion...
Tool web_search executed with result: Artificial intelligence for the discovery of novel antimicrobial agents ... - https://pmc.ncbi.nlm.nih.gov
BenevolentAI: Company Overview by Joanna Shields, CEO - YouTube - https://www.youtube.com/watch?v=...

...

[ ] Agent Final Answer

Agent: AI Researcher

Final Answer:
Atomwise: AI-driven structure-based virtual screening and small-molecule lead discovery using deep learning
for docking and binding prediction.
Exscientia: Generative-AI and active-learning platform for designing optimized small-molecule drug candidates
and accelerating lead-to-clinic timelines.
Insilico Medicine: Deep generative models and multi-omics AI for novel molecule design, target
identification, and aging-related biomarker discovery.
Recursion Pharmaceuticals: Image-based phenomics platform that combines high-throughput cellular imaging with
machine learning to discover therapeutics and repurpose compounds.
BenevolentAI: Knowledge-graph and AI-driven target identification and drug repurposing platform focused on
hypothesis generation and translational insights.

...

Spaces: 4 | () | Cell 8 of 8
```

```
main.ipynb > #
Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline ... venv (Python 3.12.3)

...
Agent: Technical Writer

Final Answer:
AI is reshaping patient diagnostics by enabling earlier, more precise, and scalable detection through pattern recognition, biomarker discovery, and multimodal data integration.

Examples and mechanisms
- Image-based phenomics (Recursion Pharmaceuticals): High-throughput cellular imaging plus machine learning extracts disease-specific phenotype signatures, supporting cellular diagnostics, patient stratification, and rapid assessment of therapeutic response.
- Multi-omics and biomarker discovery (Insilico Medicine): Deep generative models that integrate genomics, proteomics, and metabolomics accelerate identification of diagnostic biomarkers and signatures for complex diseases and aging-related conditions.
- Knowledge-graph and literature mining (BenevolentAI): AI links disparate biological data and literature to surface candidate diagnostic markers and mechanistic hypotheses, shortening the path from signal to clinical assay.
- Generative chemistry and structure-based platforms (Atomwise, Exscientia): Although focused on drug discovery, their rapid in-silico screening and design methods enable discovery of molecular probes and companion diagnostics that validate disease targets and improve diagnostic specificity.

Impact
AI delivers faster discovery of diagnostic signatures, enhanced sensitivity/specificity through multimodal fusion, and scalable screening for companion diagnostics-supporting personalized care and trial enrichment.

Challenges
Clinical validation, regulatory approval, explainability, data heterogeneity, and integration into clinical workflows remain critical barriers. Priority actions include prospective validation studies, standardized pipelines, and clinician-centric implementation to realize operational benefit.

Spaces: 4 ( ) Cell 8 of 8

main.ipynb > #
Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline ... venv (Python 3.12.3)

Task Completion

Task Completed
Name: Write a short report on how AI is transforming patient diagnostics. Limit the answer to about 200 words.
Agent: Technical Writer

...

[Parallel Result]:

AI is reshaping patient diagnostics by enabling earlier, more precise, and scalable detection through pattern recognition, biomarker discovery,

Examples and mechanisms
- Image-based phenomics (Recursion Pharmaceuticals): High-throughput cellular imaging plus machine learning extracts disease-specific phenotype
- Multi-omics and biomarker discovery (Insilico Medicine): Deep generative models that integrate genomics, proteomics, and metabolomics acceler
- Knowledge-graph and literature mining (BenevolentAI): AI links disparate biological data and literature to surface candidate diagnostic marke
- Generative chemistry and structure-based platforms (Atomwise, Exscientia): Although focused on drug discovery, their rapid in-silico screenin

Impact
AI delivers faster discovery of diagnostic signatures, enhanced sensitivity/specificity through multimodal fusion, and scalable screening for c

Challenges
Clinical validation, regulatory approval, explainability, data heterogeneity, and integration into clinical workflows remain critical barriers.

...
Spaces: 4 ( ) Cell 8 of 8
```

By following these steps, you successfully built intelligent CrewAI agent workflows that execute tasks sequentially or in parallel, structuring execution around task interdependencies and concurrency.